

NagarVaani — Project Report

Your Voice, Your City · नगरवाणी Built for GDG India — **Build with AI: Code for Communities**, Track 1 (AI for Digital Public Infrastructure & Governance).

1. What the app does (in plain words)

NagarVaani is a civic-complaint platform. A citizen reports a local problem — a blocked drain, garbage, a dead streetlight, a fallen tree, no water — by **typing, speaking (voice note) or messaging the Telegram bot**, in Hindi, Marathi, Tamil, English and more.

An AI reads each report, filters out political/communal noise, works out the category, how dangerous it is, and where it is, then gives it a **priority score out of 100**. The score is based on **severity, season, people affected and whether other areas report the same thing — not on vote count**. A 3-vote flooded drain in monsoon can outrank a 100-vote pothole.

Officials sign in (admin-approved, work email only) and see a dashboard scoped to their ward / city / state / nation: a ranked queue, a hotspot map, AI-written briefs, an *investment-flag* list, and a data-driven *project priorities* table. Citizens track the status (open → in progress → resolved) and can dispute a "resolved" complaint that isn't actually fixed.

Main features

Feature	Where
Report by web form, voice note, Telegram	Submit page · mic button · @NagarVaaniBot
AI filter + classify + priority score (L1–L5)	Backend <code>ai_engine.py</code>
Multilingual UI (English, Hindi, Marathi, Tamil...) and on-demand translation of any complaint/comment, including Hindi written in English letters (Hinglish)	Language menu · Translate button
Near-me feed with radius, heatmap, email alerts for your neighbourhood	Near Me page
Trending, votes, "same issue in my area" cross-district linking	Trending / detail page
Share a complaint (native share sheet, WhatsApp, Telegram, X, Facebook, email, copy link; opens straight to the complaint)	Detail page →  Share
Flag a complaint (spam / fake / abusive / duplicate) → moderator queue	Detail page →  Flag
Anonymous corruption reporting channel	Corruption page
Official portal: queue, hotspot map, investment flags, project priorities, verification queue (admin)	Official mode
"How scoring works (L1–L5)" explainer	Left nav → How scoring works
Government schemes + ministry directory (real links)	Schemes page
Volunteer / NGO / partner enquiry forms (email)	Get Involved

Feature	Where
Help chatbot grounded in the app's real features	Home page

2. Where does ► Flag go?

Clicking **Flag** on a complaint opens a small form (reason + optional note). It calls `POST /api/complaints/{id}/report` and writes a row in the `complaint_reports` table. Moderators (admin accounts) read the queue at `GET /api/admin/reports` and close items with `POST /api/admin/reports/{id}/resolve`.

- Flagging **never hides** the complaint automatically (so it can't be abused to silence real reports).
- One flag per signed-in user or per IP per complaint; 10 flags/hour/IP.
- **Not built yet:** an admin *screen* for this queue — today it's API-only (see §5).

(Separate thing: the ► Flag button in the official queue is the "investment flag" — an official marking an infrastructure gap for senior review. That one shows in Official → Investment Flags.)

3. What do L1–L5 mean?

Each complaint's score is built in layers. Open any complaint → "Why this scores N" to see its own numbers. There is also a public page: **How scoring works**.

Level	Meaning	Effect
L1	Immediate safety risk (someone could get hurt/sick soon)	Biggest single boost
L2	Season amplifier (blocked drain in monsoon » in winter)	Multiplies severity
L3	Problem-type base weight (drainage/water/electricity > cosmetic)	Base score
L4	Population / density (more people affected)	Adjustment
L5	Cross-district pattern (same problem in several areas = systemic gap)	Bonus
Votes	Community votes — weakest signal	Small final multiplier; can never flip a severity ranking

4. Real vs demo vs stub — honest audit

✔ Real and working (live data, real logic)

- Complaint intake (form, voice → Whisper speech-to-text, Telegram webhook) → same AI pipeline; stored in the database.
- AI filtering, classification, severity, location extraction, priority scoring, official brief (Gemini by default; switchable to Claude). Falls open safely if no API key.
- Scoring logic (L1–L5 + votes) — real code, breakdown shown per complaint is the real computed breakdown.
- Cross-district pattern linking (button + NLP place detection in comments).

- Voting, comments, status changes, disputes, notification bell (status changes on *your* complaints).
- Nationwide / ward / city / state stats and map points — computed from the actual database rows.
- Near-me: real browser geolocation + haversine distance; neighbourhood email alerts (needs SMTP).
- Translation of complaints and comments (needs the LLM key); Hinglish now detected as Hindi.
- Auth: bcrypt passwords, JWT, email OTP, Google sign-in (if client id set), rate limits, admin-approved official accounts.
- **Project priorities table** (new): uses real Census-2011 state populations and the real central scheme names.
- Share (native/social/copy link, deep link `#detail/<id>`) and Flag (persisted).
- Government schemes list and ministry links — real, India-only.

Demo / sample data (labelled or seeded — not real)

- `seed_data.py`: demo citizens (`ramesh@test.com` , `official@test.com`), ~20 "Voter N" accounts and ~5 sample complaints so the app isn't empty. **Run only for demos; don't seed production.**
- "Supporters" ticker on Get Involved: *GreenStep Foundation, Clean City Co-op, BuildRight Infra, Ward 12 Volunteers, CivicFirst Trust* — invented placeholders, ribbon-labelled DEMO.
- Static example cards on feed pages before real data loads (voting on them shows a "Demo card" toast).
- Scoring population factor is a **constant 10,000** per complaint (no per-district lookup yet).

Stubs / partly built (a provision exists, feature incomplete)

- **Our Impact**: resolved-complaint counts are real; trees planted and recycled are "Coming soon".
- **Volunteer / NGO / partner forms**: real emails to the team inbox; no partner dashboard behind them.
- **Moderation queue for 🚩 Flag**: API exists, no admin UI.
- **Infrastructure-index and public-investment-plan data**: not ingested. Only Census-2011 populations (22 states/UTs) are wired in.
- **Recommended cost / project cost**: intentionally *not* invented. Priorities table gives a triage index + suggested scheme, not a budget.
- **SMS / phone**: not implemented (no provider).
- **Countries other than India**: not implemented (no country concept in the data model).
- Voice STT on Hugging Face free tier has cold-start delays; Groq/OpenAI keys are a drop-in upgrade.

Are the clicks real?

Everything on the main flows calls the backend: vote, same-issue, comment, translate, dispute, submit, share, flag, status update, official flag, approve/reject official, subscribe, chatbot. Not real: voting on static demo cards; the Impact "coming soon" tiles.

5. Where it can grow (provisions already made)

1. **More BRICS countries** — scoring, translation and dashboards are country-agnostic; add a `country` field plus that country's schemes/ministries/languages.
 2. **Real datasets** — replace the fixed population constant with a district table (`District` model exists); ingest infrastructure indices & investment plans as CSV/API tables and join in `/stats/priorities`.
 3. **Costed project recommendations** — extend the priorities endpoint with unit-cost tables from official schedules of rates.
 4. **Moderation UI** for flags; auto-triage of duplicates using embeddings.
 5. **SMS / WhatsApp intake** through the same pipeline used by Telegram.
 6. **Open data export** (CSV/JSON, CKAN) to qualify fully as a Digital Public Good.
-

6. Fit to the GDG challenge

Challenge: a scalable, multilingual AI platform (Digital Public Good) that aggregates citizen development requests via voice, text and messaging apps; combines them with demographic data, infrastructure indices and investment plans; surfaces demand hotspots and recommends priority projects to policymakers across BRICS.

Requirement	Status
Voice, text, messaging apps	✅ web, mic, Telegram
Multilingual	✅ UI in 4 languages + LLM translation for the rest
Aggregation + hotspots	✅ live maps and state/ward rollups
Demographic data	🟡 Census-2011 state populations wired into priorities; per-complaint density still constant
Infrastructure index / investment plans	🔴 not yet — the main gap
Priority-project recommendation	🟡 transparent triage index + suggested funding scheme; no costs
Digital Public Good	🟡 MIT licence + privacy doc; open-data export pending
BRICS scope	🔴 India only (designed to extend)

Verdict: a strong, honest prototype of the intake → AI prioritisation → policymaker view pipeline. The two things that would make it convincing for the full challenge: load one real infrastructure dataset and add a second country's scheme table — both are small data additions, not re-architecture.

7. Security review

Already in place: bcrypt hashing; JWT with refuse-to-start on default secret in production; CORS locked in production; admin-approved officials with work-email check; per-IP rate limits (submit, OTP, reset, Google, uploads, chatbot); upload size/type caps; Telegram webhook secret; HTML-escaping of user text in the UI; `.env` git-ignored.

Fixed in this pass: - Security headers on every response (nosniff, X-Frame-Options DENY, Referrer-Policy, Permissions-Policy, HSTS in production). - Complaint image URLs validated as `http(s)` only (blocks `javascript:` URLs) and the gallery no longer builds inline JS from the URL (XSS). - Length caps on complaint text (5000), image count (10), URL and caption. - Rate limits on the two translation endpoints (they call a paid LLM anonymously). - Flag endpoint rate-limited and de-duplicated.

Remaining risks (be upfront about these): - Auth token lives in `localStorage` (readable if an XSS ever lands). A future step is `httpOnly` cookies. - The rate limiter is in-memory: per-process, resets on restart, not shared across workers. Use Redis when scaling. - No CSP header yet (the page uses inline handlers and CDN scripts). - Anonymous complaint submission is intentionally open — abuse is limited by rate limits and the AI filter, not identity. - Before deploy: set `ENVIRONMENT=production`, a random `JWT_SECRET`, `FRONTEND_URL`, and **change/remove the seeded test passwords**. Never commit `.env`.